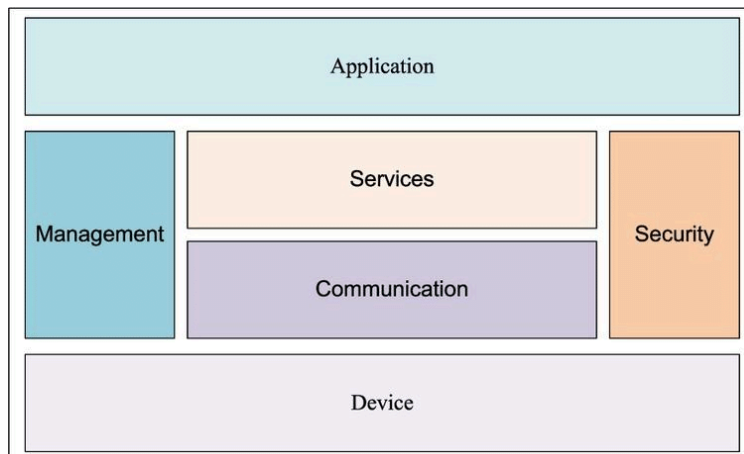


IoT Functional Blocks

1.3.1 IoT Functional Blocks

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management as shown in Figure 1.6. These functional blocks are described as follows:

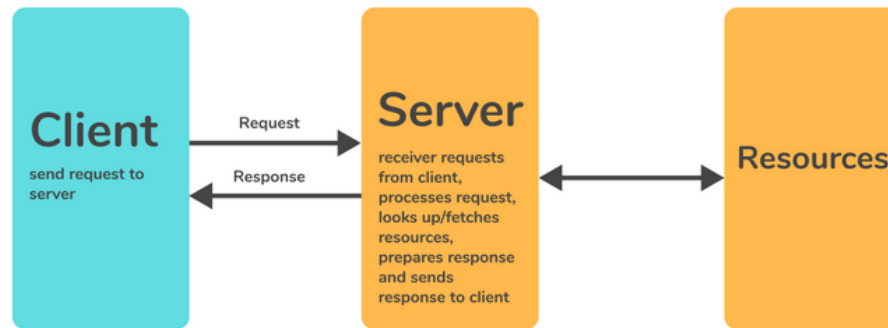
- **Device** : An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions. You learned about IoT devices in section 1.2.
- **Communication** : The communication block handles the communication for the IoT system. You learned about various protocols used for communication by IoT systems in section 1.2.
- **Services** : An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management** : Management functional block provides various functions to govern the IoT system.
- **Security** : Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application** : IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyze the processed data.



IoT Communication Models

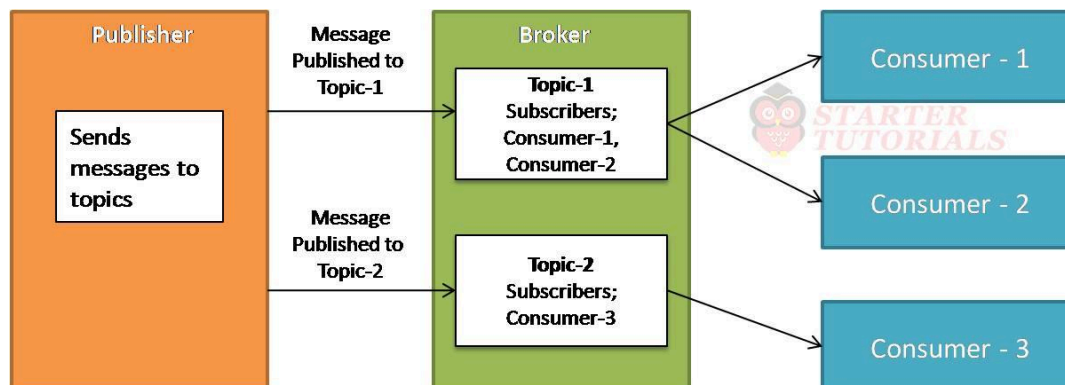
1.3.2 IoT Communication Models

- **Request-Response** : Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client. Request-Response model is a stateless communication model and each request-response pair is independent of others. Figure 1.7 shows the client-server



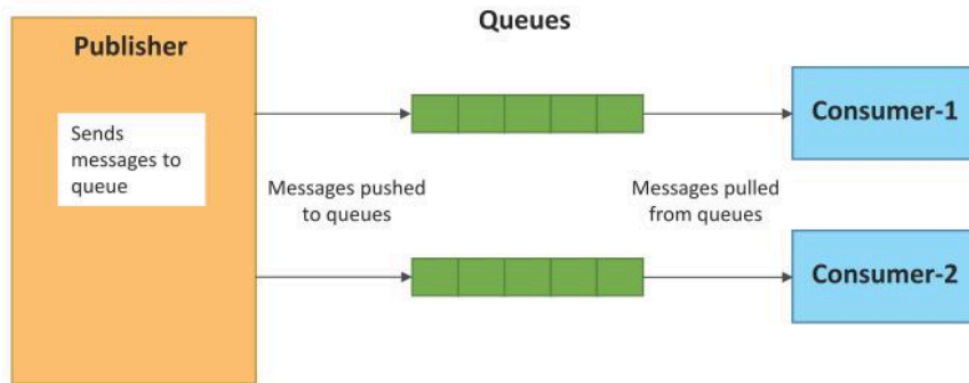
interactions in the request-response model.

- Publish-Subscribe** : Publish-Subscribe is a communication model that involves publishers, brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers. Figure 1.8 shows the publisher-broker-consumer interactions

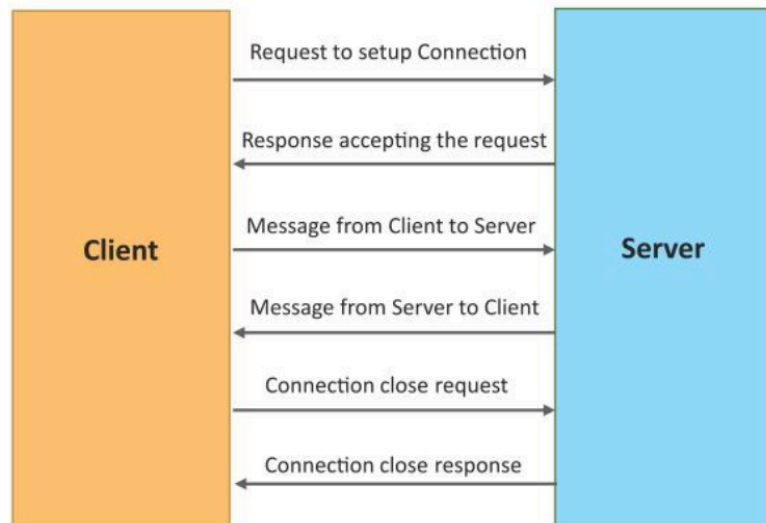


in the publish-subscribe model.

- Push-Pull** : Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the messaging between the producers and consumers. Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data. Figure 1.9 shows the publisher-queue-consumer interactions in the push-pull model.
- Exclusive Pair** : Exclusive Pair is a bi-directional, fully duplex communication model



that uses a persistent connection between the client and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Figure 1.10 shows the client-server interactions in the exclusive pair model.

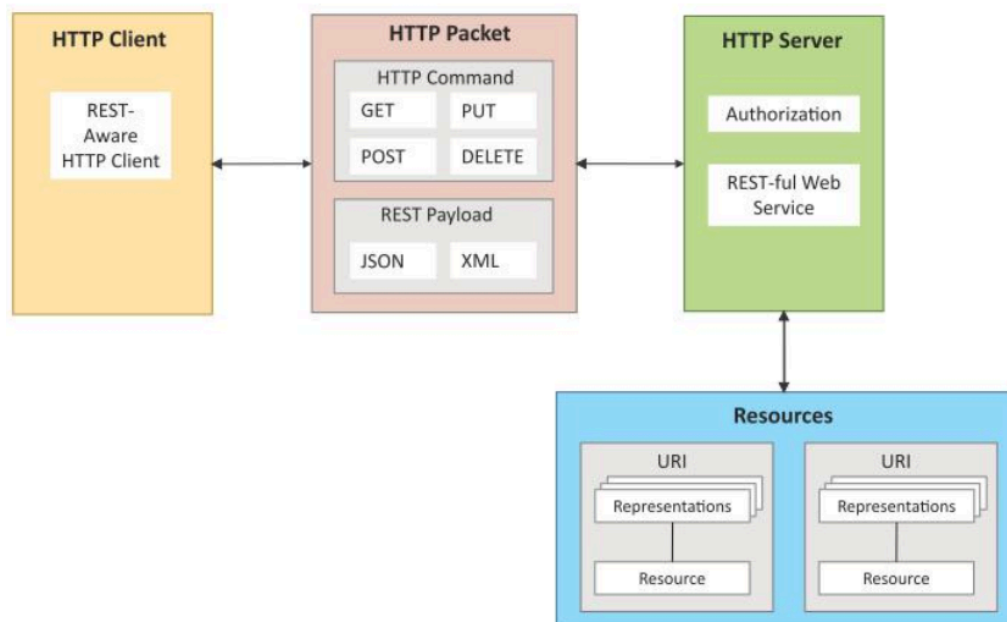


IoT Communication APIs

REST-based Communication APIs

Representational State Transfer (REST) [88] is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response communication model described in previous section. The REST architectural constraints apply to the components, connectors, and data elements, within a distributed hypermedia system. The REST architectural constraints are as follows:

- **Client-Server:** The principle behind the client-server constraint is the separation of concerns. For example, clients should not be concerned with the storage of data which is a concern of the server. Similarly, the server should not be concerned about the user interface, which is a concern of the client. Separation allows client and server to be independently developed and updated.
- **Stateless:** Each request from client to server must contain all the information necessary to understand the request, and cannot take advantage of any stored context on the server. The session state is kept entirely on the client.
- **Cache-able:** Cache constraint requires that the data within a response to a request be implicitly or explicitly labeled as cache-able or non-cache-able. If a response is cache-able, then a client cache is given the right to reuse that response data for later, equivalent requests. Caching can partially or completely eliminate some interactions and improve efficiency and scalability.
- **Layered System:** Layered system constraint, constrains the behavior of components such that each component cannot see beyond the immediate layer with which they are interacting. For example, a client cannot tell whether it is connected directly to the end server, or to an intermediary along the way. System scalability can be improved



by allowing intermediaries to respond to requests instead of the end server, without the client having to do anything different.

- **Uniform Interface:** Uniform Interface constraint requires that the method of communication between a client and a server must be uniform. Resources are identified in the requests (by URIs in web based systems) and are themselves separate from the representations of the resources that are returned to the client. When a client holds a representation of a resource it has all the information required to update or delete the resource (provided the client has required permissions). Each message includes enough information to describe how to process the message.
- **Code on demand:** Servers can provide executable code or scripts for clients to execute in their context. This constraint is the only one that is optional.

A RESTful web service is a "web API" implemented using HTTP and REST principles. Figure 1.11 shows the communication between client and server using REST APIs. Figure 1.12 shows the interactions in the request-response model used by REST. RESTful web service is a collection of resources which are represented by URIs. RESTful web API has a base URI (e.g. <http://example.com/api/tasks/>). The clients send requests to these URIs using the methods defined by the HTTP protocol (e.g., GET, PUT, POST, or DELETE), as shown in Table 1.1. A RESTful web service can support various Internet media types (JSON being the most popular media type for RESTful web services). IP for Smart Objects Alliance (IPSO Alliance) has published an Application Framework that defines a RESTful design for use in IP smart object systems [18].

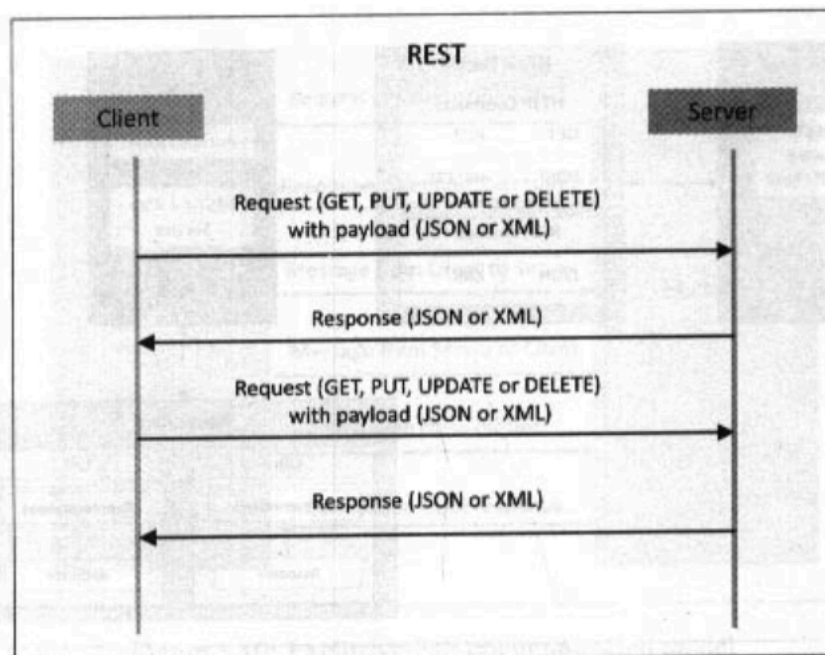


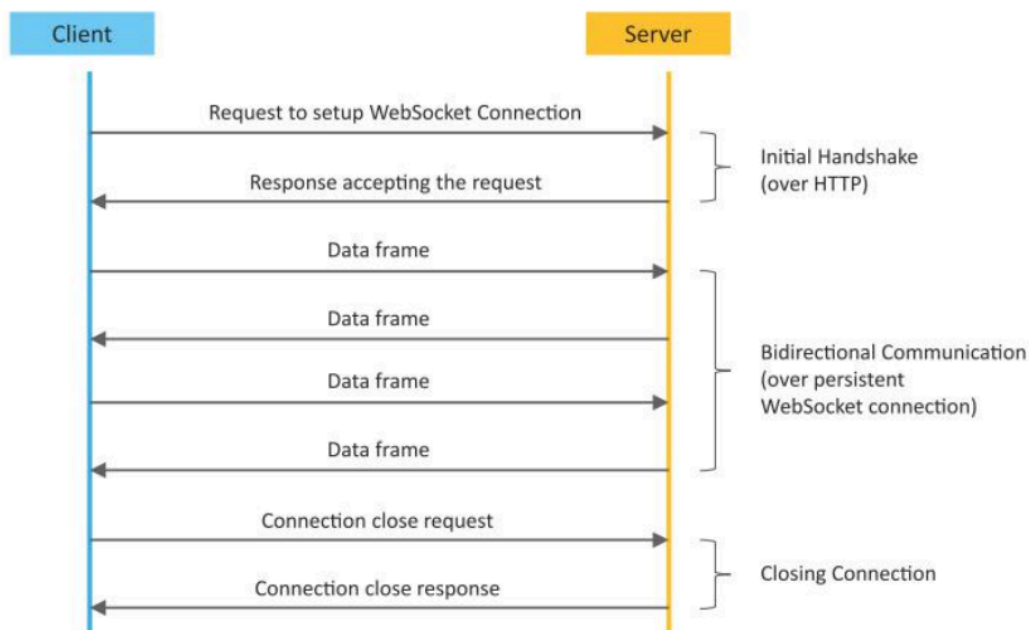
Figure 1.12: Request-response model used by REST

WebSocket-based Communication APIs

WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model described in previous section and as shown in Figure 1.13. Unlike request-response APIs such as REST, the

WebSocket APIs allow full duplex communication and do not require a new connection to be setup for each message to be sent. WebSocket communication begins with a connection setup request sent by the client to the server. This request (called a WebSocket handshake) is sent over HTTP and the server interprets it as an upgrade request. If the server supports WebSocket protocol, the server responds to the WebSocket handshake response. After the connection is setup, the client and server can send data/messages to each other in full-duplex mode. WebSocket APIs reduce the network traffic and latency as there is no overhead for connection setup and termination requests for each message. WebSocket is suitable for IoT applications that have low latency or high throughput requirements.

WebSocket Protocol



1.5 IoT Levels & Deployment Templates

In this section we define various levels of IoT systems with increasing complexity. An IoT system comprises of the following components:

- **Device:** An IoT device allows identification, remote sensing, actuating and remote monitoring capabilities. You learned about various examples of IoT devices in section 1.2.1.
- **Resource:** Resources are software components on the IoT device for accessing, processing, and storing sensor information, or controlling actuators connected to the device. Resources also include the software components that enable network access for the device.
- **Controller Service:** Controller service is a native service that runs on the device and interacts with the web services. Controller service sends data from the device to the web service and receives commands from the application (via web services) for controlling the device.
- **Database:** Database can be either local or in the cloud and stores the data generated by the IoT device.
- **Web Service:** Web services serve as a link between the IoT device, application, database and analysis components. Web service can be either implemented using HTTP and REST principles (REST service) or using WebSocket protocol (WebSocket service). A comparison of REST and WebSocket is provided below:

- **Stateless/Stateful:** REST services are stateless in nature. Each request contains all the information needed to process it. Requests are independent of each other. WebSocket on the other hand is stateful in nature where the server maintains the state and is aware of all the open connections.
- **Uni-directional/Bi-directional:** REST services operate over HTTP and are uni-directional. Request is always sent by a client and the server responds to the requests. On the other hand, WebSocket is a bi-directional protocol and allows both client and server to send messages to each other.
- **Request-Response/Full Duplex:** REST services follow a request-response communication model where the client sends requests and the server responds to the requests. WebSocket on the other hand allow full-duplex communication between the client and server, i.e., both client and server can send messages to each other independently.
- **TCP Connections:** For REST services, each HTTP request involves setting up a new TCP connection. WebSocket on the other hand involves a single TCP connection over which the client and server communicate in a full-duplex mode.
- **Header Overhead:** REST services operate over HTTP, and each request is independent of others. Thus each request carries HTTP headers which is an overhead. Due the overhead of HTTP headers, REST is not suitable for real-time applications. WebSocket on the other hand does not involve overhead of headers. After the initial handshake (that happens over HTTP), the client and server exchange messages with minimal frame information. Thus WebSocket is suitable for real-time applications.
- **Scalability:** Scalability is easier in the case of REST services as requests are independent and no state information needs to be maintained by the server. Thus both horizontal (scaling-out) and vertical scaling (scaling-up) solutions are possible for REST services. For WebSockets, horizontal scaling can be cumbersome due to the stateful nature of the communication. Since the server maintains the state of a connection, vertical scaling is easier for WebSockets than horizontal scaling.
- **Analysis Component:** The Analysis Component is responsible for analyzing the IoT data and generate results in a form which are easy for the user to understand. Analysis of IoT data can be performed either locally or in the cloud. Analyzed results are stored in the local or cloud databases.
- **Application:** IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view the processed data.

1.5.1 IoT Level-1

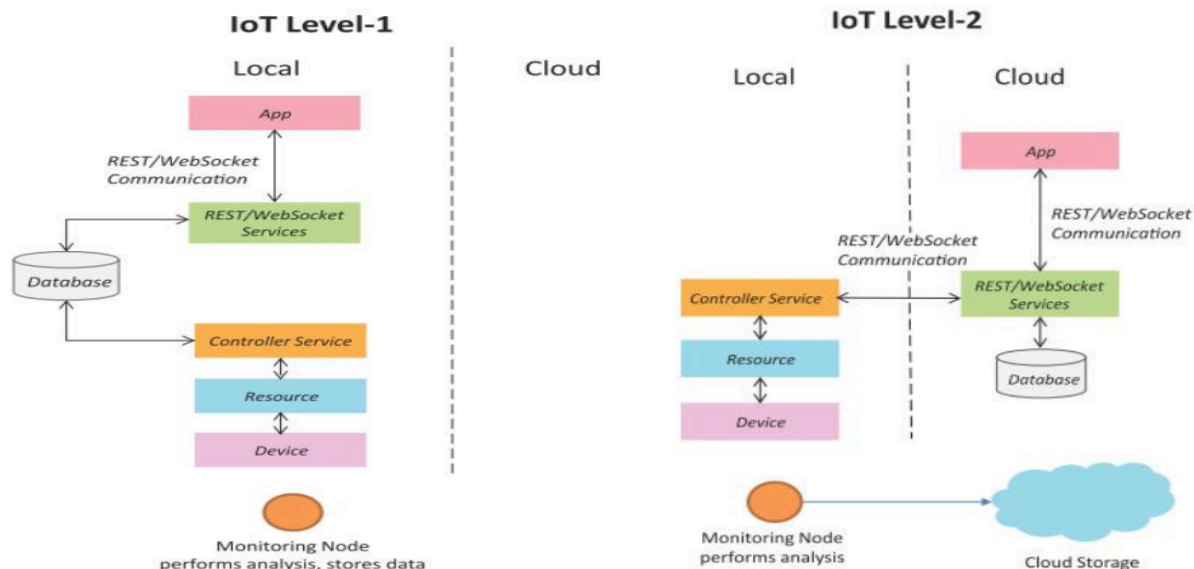
A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application as shown in Figure 1.14. Level-1 IoT systems are suitable for modeling low-cost and low-complexity solutions where the data involved is not big and the analysis requirements are not computationally intensive.

Let us now consider an example of a level-1 IoT system for home automation. The system consists of a single node that allows controlling the lights and appliances in a home remotely. The device used in this system interfaces with the lights and appliances using electronic relay switches. The status information of each light or appliance is maintained in a local database. REST services deployed locally allow retrieving and updating the state of each light or appliance in the status database. The controller service continuously monitors the state of each light or appliance (by retrieving state from the database) and triggers the relay switches accordingly. The application which is deployed locally has a user interface for controlling the lights or appliances. Since the device is connected to the Internet, the application can be accessed remotely as well.

1.5.2 IoT Level-2

A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis as shown in Figure 1.15. Data is stored in the cloud and application is usually cloud-based. Level-2 IoT systems are suitable for solutions where the data involved is big, however, the primary analysis requirement is not computationally intensive and can be done locally itself.

Let us consider an example of a level-2 IoT system for smart irrigation. The system consists of a single node that monitors the soil moisture level and controls the irrigation system. The device used in this system collects soil moisture data from sensors. The controller service continuously monitors the moisture levels. If the moisture level drops below a threshold, the irrigation system is turned on. For controlling the irrigation system actuators such as solenoid valves can be used. The controller also sends the moisture data to the computing cloud. A cloud-based REST web service is used for storing and retrieving moisture data which is stored in the cloud database. A cloud-based application is used for visualizing the moisture levels over a period of time, which can help in making decisions about irrigation schedules.



1.5.3 IoT Level-3

A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based as shown in Figure 1.16. Level-3 IoT systems are suitable for

solutions where the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-2 IoT system for tracking package handling. The system consists of a single node (for a package) that monitors the vibration levels for a package being shipped. The device in this system uses accelerometer and gyroscope sensors for monitoring vibration levels. The controller service sends the sensor data to the cloud in real-time using a WebSocket service. The data is stored in the cloud and also visualized using a cloud-based application. The analysis components in the cloud can trigger alerts if

the vibration levels become greater than a threshold. The benefit of using WebSocket service instead of REST service in this example is that, the sensor data can be sent in real time to the cloud. Moreover, cloud based applications can subscribe to the sensor data feeds for viewing the real-time data.

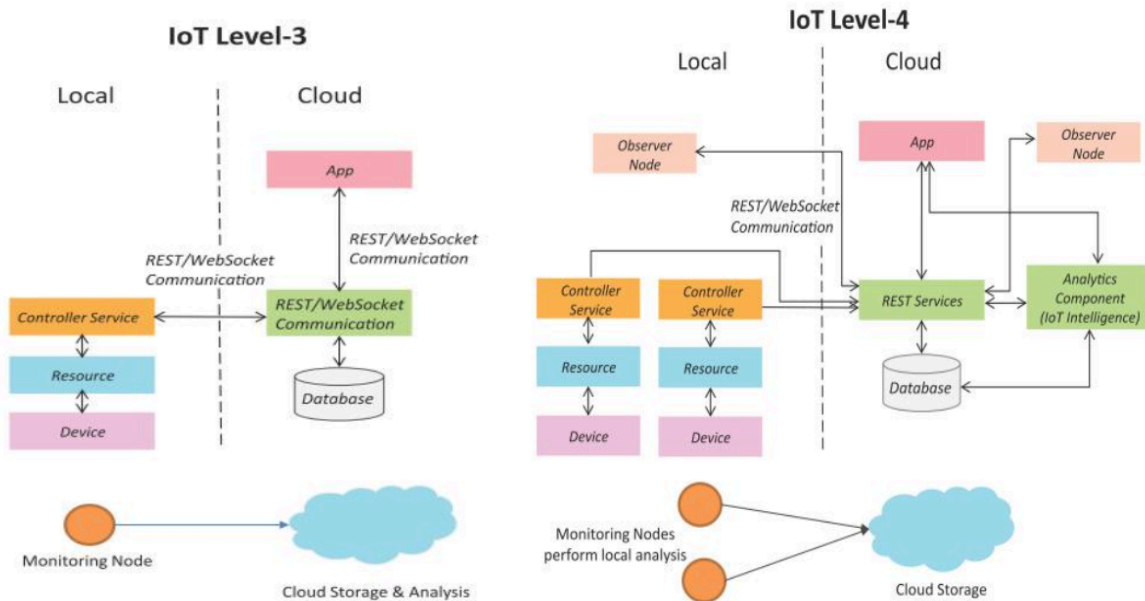
1.5.4 IoT Level-4

A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based as shown in Figure 1.17. Level-4 contains local and

cloud-based observer nodes which can subscribe to and receive information collected in the cloud from IoT devices. Observer nodes can process information and use it for various applications, however, observer nodes do not perform any control functions. Level-4 IoT systems are suitable for solutions where multiple nodes are required, the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-4 IoT system for noise monitoring. The system consists of multiple nodes placed in different locations for monitoring noise levels in an area.

The nodes in this example are equipped with sound sensors. Nodes are independent of each other. Each node runs its own controller service that sends the data to the cloud. The data is stored in a cloud database. The analysis of data collected from a number of nodes is done in the cloud. A cloud-based application is used for visualizing the aggregated data.



1.5.5 IoT Level-5

A level-5 IoT system has multiple end nodes and one coordinator node as shown in Figure 1.18. The end nodes that perform sensing and/or actuation. Coordinator node collects data from the end nodes and sends to the cloud. Data is stored and analyzed in the cloud and application is cloud-based. Level-5 IoT systems are suitable for solutions based on wireless sensor networks, in which the data involved is big and the analysis requirements are computationally intensive.

Let us consider an example of a level-5 IoT system for forest fire detection. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and carbon dioxide (CO_2) levels in a forest. The end nodes in this example are equipped with

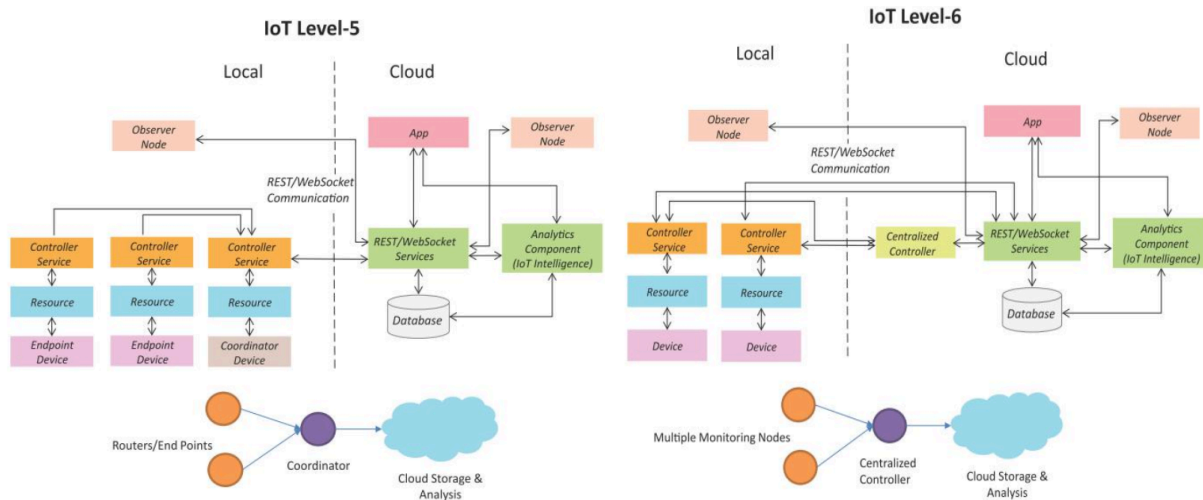
various sensors (such as temperature, humidity and CO_2). The coordinator node collects the data from the end nodes and acts as a gateway that provides Internet connectivity to the IoT system. The controller service on the coordinator device sends the collected data to the cloud. The data is stored in a cloud database. The analysis of data is done in the computing cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.

1.5.6 IoT Level-6

A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud. Data is stored in the cloud and application is cloud-based as shown in Figure 1.19. The analytics component analyzes the data and stores the results in the cloud database. The results are visualized with the cloud-based application. The centralized controller is aware of the status of all the end nodes and sends control commands

to the nodes.

Let us consider an example of a level-6 IoT system for weather monitoring. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and pressure in an area. The end nodes are equipped with various sensors (such as temperature, pressure and humidity). The end nodes send the data to the cloud in real-time using a WebSocket service. The data is stored in a cloud database. The analysis of data is done in the cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data.



IoT Design Methodologies (for a particular problem statement)

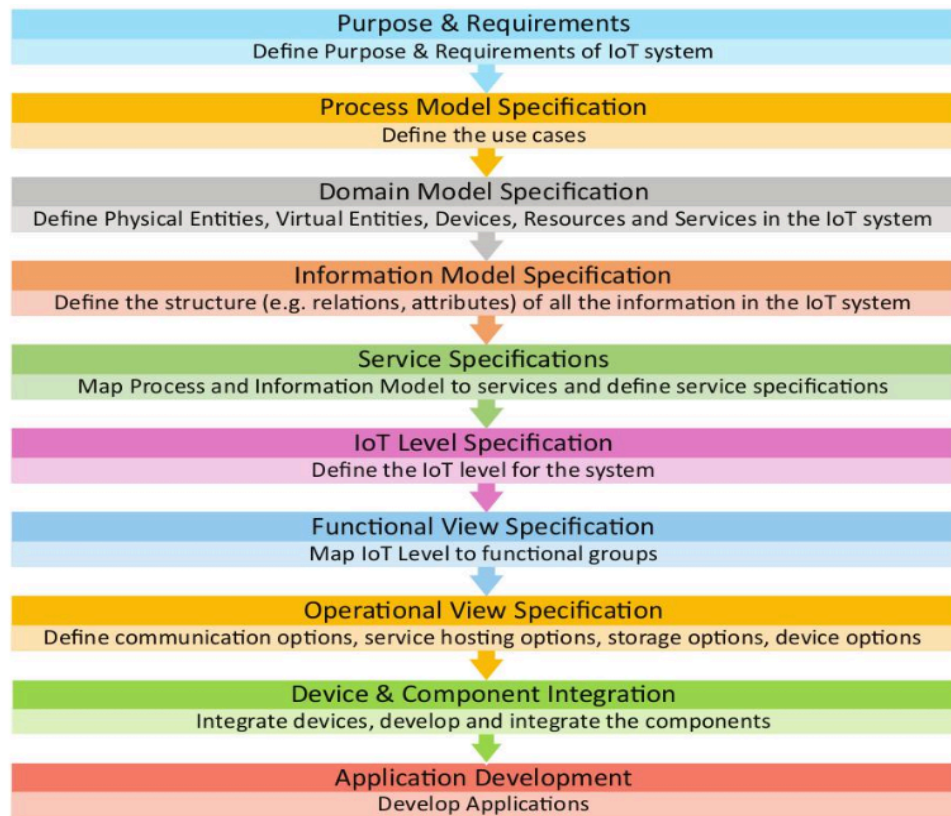
Step 1: Purpose & Requirements Specification: The first step in IoT system design methodology is to define the purpose and requirements of the system. In this step, the system purpose, behavior and requirements (such as data collection requirements, data analysis requirements, system management requirements, data privacy and security requirements, user interface requirements, ...) are captured.

Step 2: Process Specification: The second step in the IoT design methodology is to define the process specification. In this step, the use cases of the IoT system are formally described based on and derived from the purpose and requirement specifications.

Step 3: Domain Model Specification: The third step in the IoT design methodology is to define the Domain Model. The domain model describes the main concepts, entities and objects in the domain of the IoT system to be designed. The domain model defines the attributes of the objects and relationships between objects. Domain model provides an abstract representation of the concepts, objects and entities in the IoT domain, independent of any specific technology or platform. With the domain model, the IoT system designers can get an understanding of the IoT domain for which the system is to be designed.

Step 4: Information Model Specification: The fourth step in the IoT design methodology is to define the Information Model. Information Model defines the structure of all the information in

the IoT system, for example, attributes of Virtual Entities, relations, etc. Information model does not describe the specifics of how the information is represented or stored. To define the information model, we first list the Virtual Entities defined in the Domain Model. Information model adds more details to the Virtual Entities by defining their attributes and relations.



Step 5: Service Specifications: The fifth step in the IoT design methodology is to define the service specifications. Service specifications define the services in the IoT system, service types, service inputs/output, service endpoints, service schedules, service preconditions and service effects.

Step 6: IoT Level Specification: The sixth step in the IoT design methodology is to define the IoT level for the system.

Step 7: Functional View Specification: The seventh step in the IoT design methodology is to define the Functional View. The Functional View (FV) defines the functions of the IoT systems grouped into various Functional Groups (FGs). Each Functional Group either provides functionalities for interacting with instances of concepts defined in the Domain Model or provides information related to these concepts.

Step 8: Operational View Specification: The eighth step in the IoT design methodology is to define the Operational View Specifications. In this step, various options pertaining to the IoT system deployment and operation are defined, such as, service hosting options, storage options, device options, application hosting options, etc.

Step 9: Device & Component Integration: The ninth step in the IoT design methodology is the integration of the devices and components.

Step 10: Application Development: The final step in the IoT design methodology is to develop the IoT application.

Real-world design constraints

Devices and networks

- The devices that form networks in the M2M Area Network domain must be selected, or designed, with certain functionality suitable to IoT applications.
- The devices must have an energy source (e.g. batteries), computational capability (e.g. an MCU), appropriate communications interface (e.g. a Radio Frequency Integrated Circuit (RFIC) and front end RF circuitry), memory (program and data), and sensing (and/or actuation) capability.
- These must be integrated in such a way that the functional requirements of the desired application can be satisfied with additional nonfunctional requirements.

Functional Requirements: Specific sensing and actuating capabilities

- Sensing principle and data requirements: Sometimes continuous sampling of sensing data is required. For some applications, sampling after specific intervals is required.
- The parameters like higher network throughput, data loss, energy use, etc are decided based on sensing principle.

Sensing and communications field

- The sensing field is to be considered for sensing in local areas or distributed sensing. The distance between sensing points is also an important factor to be considered.
- The physical environment has an implication on the communications technologies selected and the reliability of the system in operation thereafter.
- Devices must be placed in close enough proximity to communicate. Where the distance is too great, routing devices may be necessary.

Programming and embedded intelligence

- Devices in the IoT are heterogeneous such as various computational architectures, including MCUs (8-, 16-, 32-bit, ARM, 8051, RISC, Intel, etc.), signal conditioning (e.g. ADC), and memory (ROM, S/F/D) RAM, etc.), communications media, peripheral components (sensors, actuators, buttons, screens, LEDs), etc.
- In every case, an application programmer must consider the hardware selected or designed, and its capabilities.
- Application-level logic decides the sampling rate of the sensor, the local processing performed on sensor readings, the transmission schedule (or reporting rate), and the management of the communications protocol stack, among other things.
- The programmers have to reconfigure and reprogram devices in case of change in devices in IoT applications.

Power

- Power is essential for any embedded or IoT device.
- Depending on the application, power may be provided by the mains, batteries, or hybrid

power sources.

- Power requirements of the application are modeled prior to deployment. This allows the designer to estimate the cost of maintenance over time.

Gateway: Gateway devices or proxies are selected according to the need of data transitions.

Nonfunctional requirements

The non-functional requirements are technical and non-technical.

- Regulations:
 - For applications that require placing nodes in public places, prior permissions are important.
 - Radio Frequency (RF) regulations limit the power with which transmitters can broadcast.
- Ease of use, installation, maintenance, accessibility:
 - This relates to positioning, placement, site surveying, programming, and physical accessibility of devices for maintenance purposes.
- Physical constraints:
 - Integration of additional electronics into existing system
 - Suitable packaging
 - Kind and size of antenna
 - Type of power supply

Financial cost: Financial cost considerations are as follows:

- Component Selection: Typically, the use of these devices in the M2M Area Network domain is to reduce the overall cost burden. However, there are research and development costs likely to be incurred for each individual application in the IoT that requires device development or integration. Developing devices in small quantities is expensive.
- Integrated Device Design: Once the energy, sensors, actuators, computation, memory, power, connectivity, physical, and other functional and nonfunctional requirements are considered, it is likely that an integrated device must be produced.

Data representation and visualization:

Each IoT application has an optimal visual representation of the data and the system. Data that is generated from heterogeneous systems has heterogeneous visualization requirements. There are currently no satisfactory standard data representation and storage methods that satisfy all of the potential IoT applications.

Case study on IoT system

Any topic can be asked (open-ended question). Follow the given template for answering:

1. Introduction (Context, Know the user, Use cases, Components)
2. Objective
3. Architecture/methodology/protocols
4. Result

For example: a smart irrigation system.